

KAIROX ERP · PHASE 1

The Import Service

Excel breakdown sheet → barcoded garments.

Every file, every function, every hand-off between modules.

Read this first — the whole module in six lines

A client emails a spreadsheet. The Direct Manager types the order number, uploads the file, looks at what the parser found, fixes what the sheet got wrong, and then presses Release. Only at that last press does the factory create real, permanent, printed barcodes.

```

UPLOAD      →    PREVIEW      →    COMMIT      →    CORRECT      →    RELEASE
the .xlsx   read-only   styles + SKUs → DM edits   MINT barcodes
              nothing is  stored as   the draft   PERMANENT,
              written     DRAFT      table       printed on leather
  
```

The single most important rule in this module

Upload writes nothing physical. Release does.

A commit stores styles and SKUs as DRAFT — rows on a screen, deletable, editable, harmless. Release mints *Piece* rows, their per-piece barcodes, and their drawers. Barcodes get printed and stuck on leather; they are permanent garment identities and there is no undo.

That is why the flow has two phases instead of one, and why every edit endpoint returns **409** the moment a style is RELEASED.

The 10 files, in the order data moves through them

#	FILE	ROLE IN THE PIPELINE
1	router.py	HTTP only. Validates the upload stream , checks the role and the order number, hands off to a worker thread.
2	excel_reader.py	Reads one cell safely. Cleans strings, coerces ints / dates / money. Nothing else touches a raw cell.
3	_size_band.py	Finds which columns are sizes by arithmetic , not by a word list — and names the tail by cell shape. The cleverest file here.
4	parse_orders.py	Turns an ORDER sheet into OrderLine rows and returns the verdict on whether it is an order sheet at all.
5	import_engine.py	Opens the workbook, parses every sheet once , builds the ImportPreview.
6	load_to_db.py	Writes a preview into one order: Client → Order → Style → SKU → SkuOrderLine.
7	breakdown.py	The editable table, and the audited release gate. This is where the mint is triggered.
8	premint.py	The mint itself: Piece + 2 barcodes each + a recycled drawer.
9	service.py	A thin facade kept for API stability. The router bypasses it today.
10	__init__.py / README.md	Why the module exists: purchase orders arrive as spreadsheets forever, so this is not a one-time migration.

What changed in this edition. `parse_production.py` has been deleted — the importer reads **order sheets only**, and weekly production sheets are reported and skipped. Sheet classification is no longer a separate pass: parsing returns its own verdict. Price and delivery are now identified by the *shape of their cells* rather than by header names. The router no longer writes a temp file. Each of these is explained where the function is documented.

1 The end-to-end flow, from email attachment to barcode

Stage A — PREVIEW (a dry run; the database is never touched)

```
POST /api/v1/imports/preview      (multipart: order_number + file)

router._require_order(db, no)      ClientService.get_order_by_number → 404 if unknown
router._validated_upload(file)     extension → size cap → IS IT A REAL ZIP?
run_in_threadpool(_do_preview)     openpyxl + parsing are BLOCKING; keep the loop free

import_engine.build_preview(stream)
  for each sheet:
    _client_key(sheet_name)        which client does this sheet belong to
    parse_order_sheet(ws)          ONE call: returns (lines, warnings, verdict)
    ImportPreview.summary()         clients, styles, sizes, prices, warnings

no temp file to clean up           the upload's own spooled stream was used
```

The response is what a human checks: how many pieces per style per size, the unit price and delivery date the sheet printed, and every warning the parsers raised.

Two things here are new. The handler is `_validated_upload`, not `_save_upload`: Starlette has already spooled the upload to a seekable file and `openpyxl` reads a file-like object, so copying the bytes to a second file on disk bought nothing. All three guards survived the change — extension, size cap, zip-container check.

And there is no `_sheet_type()` call any more. Classification and parsing are the same pass; see §3.4.

Stage B — COMMIT (writes DRAFT rows; still mints nothing)

```
POST /api/v1/imports/commit      (multipart: order_number + file)

same guards, then run_in_threadpool(_do_commit_into_order)

build_preview(stream)             parsed exactly as in the dry run
load_preview_into_order(db, preview, order_number)
  └ advisory lock → history guard → replace → styles → SKUs → order lines

response: release_required = True, pieces_minted = 0
```

Stage C — RELEASE (the irreversible one)

```
POST /api/v1/imports/breakdown/{order_number}/release
body: { "styles": [{ "style_id": "...", "needs_lining": true }], ... }

BreakdownService.release_styles()
  per style, cheapest check first → each failure lands in `rejected`, the rest continue
  1. does the style belong to this order?
  2. is it still DRAFT?              (already RELEASED = nothing new to mint)
  3. does it have qty_ordered > 0?
  4. StyleSpecService.blockers_for_styles()  material spec confirmed?

run_in_threadpool(_release_sync)  SYNC session, ONE transaction
  stamp Style.needs_lining from the DM's answer ← FIRST, then db.flush()
  premint_order(db, order, style_ids=[...])     ← reads that flag per piece
  stamp production_status=RELEASED, released_at, released_by
  db.commit()                                   atomic: mint + status together

write one audit_log row per style, then commit the async session
```

Why the lining answer is written before the mint, not after. Pre-mint reads `Style.needs_lining` to set each new piece's own flag. Writing the declaration afterwards would mint every piece with the wrong flag and then quietly disagree with its own style.

2 Which other modules the import service touches

The import module owns no tables of its own. It is a writer into other modules' tables and a caller of other modules' services. Every arrow below is a real import statement in the code.

Modules the importer CALLS OUT to

FROM (FILE)	TO (MODULE)	WHAT IT NEEDS IT FOR
router.py	clients.service	<code>get_order_by_number()</code> — reject an unknown order number before a single cell is parsed.
router.py	users.deps	<code>require_roles(DIRECT_MANAGER, MANAGING_DIRECTOR)</code> — upload, correct and release are all DM work; MD passes as superuser.
load_to_db.py	clients.models + utlis	<code>Client</code> , <code>ClientOrder</code> , <code>Style</code> , <code>SKU</code> , <code>SkuOrderLine</code> ; <code>make_style_code()</code> and <code>make_sku_code()</code> build the deterministic codes.
load_to_db.py	production.models	<code>ProductionEvent</code> only , and only to check that no floor history exists before a replace. <i>The Operation import is gone with the production-card path.</i>
load_to_db.py	wages.models	<code>Rate</code> — imported only to delete stale rates when an order is replaced. <i>The importer no longer creates wage rates.</i>
breakdown.py	materials.style_spec_service	<code>blockers_for_styles()</code> — the material-spec release gate: no confirmed recipe, no release.
breakdown.py	core.lining_rules	<code>lining_required()</code> — one definition of “does this garment take a lining?”, shared with the drawer gate.
breakdown.py	production.models	<code>Piece</code> — counts minted garments per style for the table and the order index.
premint.py	barcode.models + repository	<code>BarcodeRegistry</code> , <code>Drawer</code> ; <code>encode_short()</code> / <code>decode_short()</code> for the compact PC-XXXXXX code.
premint.py	production.models	<code>Piece</code> — the row this whole module exists to create.

The data hierarchy the importer fills in

Client	<code>_get_or_create_client</code>	name is the natural key
└ ClientOrder	MUST already exist	commit 404s, it never creates one
└ Style	<code>_get_or_create_style</code>	code = order · style · article
└ SKU	<code>_upsert_sku</code>	one row per (style, colour, size)
└ SkuOrderLine	the audit trail: which sheet row, what date, what qty	code = order · style · colour · size · article
└ Piece	← MINTED AT RELEASE ONLY, never at upload	
└ BarcodeRegistry	PC-XXXXXX compact, PRIMARY, carries order/sku/style FKs	
└ BarcodeRegistry	long code ALIAS, is_alias=True, carries NO FKs	
└ Drawer	taken from the recycling pool	

The call-flow map — one upload, every function, in order

If you meet a function cold anywhere in §3, find it here first: this is where it sits in the journey.

```

POST /imports/preview (or /commit)
├─ require_roles(DIRECT_MANAGER)
├─ _require_order(order_number)
├─ _validated_upload(file)
├─ run_in_threadpool → _do_preview / _do_commit_into_order
├─ build_preview(source) import_engine.py
│   └─ openpyxl.load_workbook(data_only=True)
│       └─ for each sheet:
│           └─ _client_key(sheet_name)
│               └─ parse_order_sheet(ws) parse_orders.py
│                   └─ _is_header_row(ws, r) split stacked blocks
│                       └─ _block_rows(header) / close_block()
│                           └─ find_header_row(ws) only if no words found
│                               └─ _find_columns(ws, header, rows)
│                                   └─ detect_size_band() _size_band.py
│                                       └─ _numeric_count() pick the profile rows
│                                           └─ _looks_like_a_size_label()
│                                               └─ _is_quantity_column()
│                                                   └─ _qty() / _row_sum() the arithmetic
│                                                       └─ _classify_tail() price + delivery, by shape
│                                                           └─ _fallback_band() only if nothing reconciles
│                                           └─ clean_str / to_int / to_date / to_money excel_reader.py
│                                               └─ _sheet_names_itself(ws) the strict word test
│                                                   └─ (lines, warnings, verdict)

COMMIT ONLY:
├─ load_preview_into_order() load_to_db.py
│   └─ _order_lock_key() → _acquire_order_import_lock()
│       └─ _assert_no_production_history()
│           └─ _get_or_create_client / _get_or_create_order / _get_or_create_style
│               └─ _apply_style_commercials()
│                   └─ _upsert_sku() STOPS HERE. Nothing minted.

RELEASE ONLY:
├─ BreakdownService.release_styles() breakdown.py
│   └─ _release_sync()
│       └─ premint_order() premint.py
│           └─ _sku_needs_lining()
│               └─ _PoolAllocator.take()
│                   └─ _caption_prefix()
│                       └─ Piece + 2 barcodes + a Drawer. PERMANENT.

```

3 File by file, function by function

Every function in the module is here. The parsing files carry a worked example against a real sheet in `data/`, because the arithmetic only becomes obvious when you can see the cells it ran on.

3.1 excel_reader.py — the only place that touches a raw cell

Real spreadsheets are messy: trailing spaces, numbers stored as text, merged cells, stray formatting. Every other layer reads cells through these helpers, so the mess is handled in exactly one place.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>clean_str(value)</code>	Purpose: turn a cell into tidy text, or None if it is empty. How: trim the ends, return None if nothing is left, then collapse any run of spaces inside to one. Why it matters: ' CARNABY ' and 'CARNABY' must be <i>one</i> style, not two — otherwise the order silently splits in half.
<code>to_int(value)</code>	Purpose: turn a cell into a whole number, or None. How: booleans are rejected on purpose (in Python True is 1, and a TRUE cell is not a quantity of one); a real number passes through; NaN returns None; text is accepted only if it looks exactly like 12 or 12.0. Why: Excel stores a quantity as the float 12.0 or the text '12' unpredictably. Returning None for everything else is how a text column gets told apart from a size column.
<code>to_date(value)</code>	Purpose: turn a cell into a calendar date, or None. How: a real datetime cell passes through; otherwise try dd/mm/yyyy, dd-mm-yyyy, yyyy-mm-dd, dd/mm/yy in turn. Why it matters downstream: this is half of how the tail is classified — a column whose cells parse as dates is DELIVERY. <code>to_date('83')</code> returning None is exactly what stops a price column being mistaken for one.
<code>to_money(value)</code>	Purpose: turn a price cell into (amount, currency). Never raises. How: find a currency mark; strip trade terms (cif, fob, per pc); strip everything that is not a digit, dot, comma or minus; then apply the separator rule below. Why this is not to_int with a division: a price cell carries three kinds of mess a quantity cell never does — a currency mark (the only place the sheet says what money it quotes), European decimals, and trade terms stapled on the end.
the separator rule	Whichever of . or , appears last is the decimal point; the other is thousands. 1.234,50 and 1,234.50 both become 1234.50 — true in both conventions, no locale setting needed. A lone separator is a decimal only when exactly two digits follow it. Guessing wrong here is a 1000x costing error on every garment.
<code>looks_like_week_period(t)</code>	True if the text holds a date range like 14/01/2026 TO 22/01/2026. Status: no longer reached by the import pipeline — it served the production-sheet parser. Kept because other modules may import it.
<code>is_summary_label(t)</code>	Classifies a label cell as QTY / RATE / TOTAL / OVERALL, or None. Status: no longer reached by the import pipeline , same reason.

Worked example — to_money, on the three shapes that actually arrive

```
'€ 78,50 cif' → (Decimal('78.50'), 'EUR')    KairoX template, row 5
              € recognised                    → EUR
              'cif' stripped as a trade term
              one comma, two digits after it → a decimal point

'1.234,50'   → (Decimal('1234.50'), None)    comma last → comma is the decimal
'1,234.50'   → (Decimal('1234.50'), None)    period last → period is the decimal
83           → (Decimal('83.00'), None)     a plain number needs no guessing
```

3.2 _size_band.py — finding the size columns by arithmetic

The problem this file solves

The old detector asked “is this header one of XS/S/M/L/XL?”. That works until a brand ships Italian 38–62, French 36–52, a combined 48/M label or a Japanese LL — then the column is silently dropped and the order ships short. Worse, it also says yes to any stray integer header (a year, an item code), inventing a size out of a decoy.

A maintained word list is a promise to edit code every time the factory wins a client in a new country. **This file removes the promise.**

Every breakdown sheet ever sent here has the same shape, and the sheet prints its own answer:

```
[ descriptor text ]   [ SIZE BAND: one column per size ]   [ tail: total / price / delivery ]

so the band is not RECOGNISED, it is SOLVED FOR:

    find a total column T and a contiguous run of quantity columns [i..j], j < T,
    such that for EVERY data row:      sum(row[i..j]) == row[T]
```

A run that satisfies that across a whole sheet is the size band, whatever the columns are called. A decoy integer column cannot join it (it would break the sum) and cannot be mistaken for it (it sums to nothing).

FUNCTION	WHAT IT DOES / HOW IT WORKS
SizeBand (dataclass)	The answer sheet for one block: size_cols {col: verbatim label}, total_col, price_col, delivery_col, confidence, reconciled_rows, mismatched_rows, and warnings (new — carries the tail notes up into the preview). first_col is a property: the leftmost size column, so everything left of it is descriptor.
_norm(t)	clean_str then upper-case. Why: 'XL ', 'x1' and 'XL' must be ONE size — three would mean three SKUs, three codes and three barcodes for one garment. It folds case and spacing only: 'XXL/54' stays 'XXL/54', because whatever the client wrote is what the floor reads back.
is_price_header(t) is_delivery_header(t)	Small alias sets naming the tail columns (PREZZO, PRIX, CONSEGNA, SHIP DATE...). Status changed — these are no longer the decision. They used to <i>be</i> how price and delivery were found. They are now a tiebreaker for a column whose cells are ambiguous, and a cross-check that raises a warning when the name and the cells disagree. See _classify_tail.
_looks_like_a_size_label(t)	Purpose: could this header be a size? A test of shape, never of vocabulary. How: not empty, at most 8 characters, no decimal point, and if it is all digits then under 1000. Why: this is the one thing keeping a costing sheet out — see the box overleaf. A new country using 72, or 4XL, or 48/M passes through untouched.
_qty(ws, r, c)	One cell as a quantity, or None. A one-line wrapper over to_int, named because the search below calls it thousands of times and the arithmetic has to stay readable.
_is_quantity_column(ws, profile_rows, c)	Purpose: does this column hold numbers (a size) or words (a label)? How: count numeric against text cells across the profile rows; blanks are skipped; an entirely empty column still qualifies (size 62 printed, none ordered — the label is real). A majority, not a purity: numeric cells must merely <i>outnumber</i> text cells. Tolerating a stray GRAND TOTAL caption inside a size column is what fixed a 146-piece order that reported 65.
_row_sum(ws, r, cols)	Add one row across a range of columns, counting blanks as zero. Why blanks are zero: most size cells are empty. John Peter's CLERMONT row orders only 48–54 out of a printed 38–62, so eleven of the sixteen columns are blank. Blank means none of that size.
_classify_tail(...) NEW	Purpose: of the columns right of the sizes, which is the price and which is the delivery date. How: the tail is every labelled column right of the band minus the total (that part is structural); for each, count cells that parse as dates against cells that parse as money; more dates means DELIVERY, otherwise any money means PRICE; compare with what the header claims and on disagreement trust the cells and warn ; finally warn if the 95% layout (TOTAL, then PRICE, then DELIVERY) was not followed. Why it runs after the band: the tail is <i>defined</i> as everything right of the band, so it cannot exist until the band is known.
detect_size_band(header_row, data_rows)	The solver, for one block. (1) pick the profile rows; (2) provisionally exclude any column openly headed PRICE or DELIVERY so it cannot be drafted into the band; (3) shortlist eligible columns; (4) try every candidate total column, rightmost first; (5) try every contiguous run to its left; (6) a run wins only if every checkable row reconciles; (7) and the run must carry a positive quantity ; (8) prefer the widest winner, since a narrower one only wins by dropping columns that happened to be empty; (9) name the tail and return HIGH; (10) otherwise hand off to _fallback_band.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>_fallback_band(...)</code>	When nothing reconciles: take the longest contiguous run of quantity columns outside the tail and mark it MEDIUM. Two different sheets land here and must not be reported alike — one printed no total at all (a reasonable guess; a human should confirm it), the other printed a total that does not add up (a torn or hand-edited sheet, which is the case the arithmetic exists to catch, so the disagreeing rows are named: <i>Row 14: sheet says 30, sizes sum to 27</i>). Never HIGH.
<code>find_header_row(ws)</code>	Purpose: find the header on a sheet whose words we do not recognise, by looking at what comes <i>after</i> each candidate row. How: walk rows top-down, skip any row labelling fewer than 3 columns, try each as a header, and require a band at least 2 columns wide. First HIGH wins; otherwise the first MEDIUM. Why it skips a title row naturally: nothing under “MAIN SS27” reconciles, because the row beneath it is the real header and holds text, not quantities.

Worked example — the GGZ order, and the column that used to go missing

```

row 2  S.NO | STYLE | COLOUR | 44 | 46 | 48 | 50 | 52 | 54 | TOTAL
row 3    1 | BOMBER A | MORO | 2 | 23 | 40 | 43 | 28 | 10 | 146
row 5    | | | | | | | | | 146 ← a subtotal footer
row 11   | | | | | | | GRAND TOTAL | 146 ← caption, col 7
          ^^^^^^^^^^^^^^
          that caption sits in the size-50 column

profile rows = [3]      col 7 sees 43      → numeric 1, text 0 → a size
if 5 and 11 joined in  col 7 sees 43, 'GRAND TOTAL' → numeric 1, text 1 → DROPPED

dropping col 7 splits the run 4-10 into [4,5,6] and [8,9,10]
the band becomes 44|46|48 → 2+23+40 = 65 reported for a 146-piece order

SOLVED: try total_col = 10, run 4..9
        2+23+40+43+28+10 = 146 == cell(3,10)    every checkable row matches,
                                                and the sum is > 0 → HIGH

```

The zero rule — why a band must carry a positive quantity.

$0 + 0 + 0 == 0$ is true, and it proves nothing. NIPAL-NEW PRODUCTION has stage columns that are zero for seven straight rows, so a run of them reconciles perfectly against an all-zero total. That was accepted at **HIGH** confidence: a *data* row's literal values were read as column headings, and a weekly progress sheet came back classified ORDER, parsed to zero pieces, with no warning at all.

The profiling rule — why `profile_rows` wants rows with 2+ numbers, not 1.

`profile_rows` is **not** a selection of rows to parse; every row is parsed later regardless. It is the sample the *columns* are judged over. Widening it to 1 lets footer rows in, and a footer caption inside the band disqualifies a real size column. Measured cost of getting this wrong: **GGZ 146 → 65, NIPAL 259 → 206, RICANO 150 → 109** — about 185 pieces, silently. The inner helper `_numeric_count(r)` draws the line: a data row carries several numbers, a footer carries one.

Why `_looks_like_a_size_label` caps the number — the costing sheet.

SAMPLE COSTING SHEET FOR SIR.xlsx has a run of columns headed **576000 / 0.05 / 28800** that reconciles across 33 rows. Arithmetically it is a perfect size band, and on the numbers alone it is indistinguishable from a breakdown. It is only implausible once you look at what the columns are **called**. Garment sizes run to about 70 and never carry a decimal point. Without this test the workbook came back as a **16,649,667-piece order**.

Why the tail is read by shape — both layouts are real.

The KairoX template prints ... | XXXL | TOTAL | PRICE | DELIVERY. John Peter prints ... | 62 | Prezzo (€) | Totale Capi — **price before the total**. About 95% of sheets are the first shape, but John Peter is a real client who will not be changing their spreadsheet, so a rule of “price comes after the total” would misread every sheet they have ever sent. Names cannot decide it either: no list knows *Precio Unitario* or *Costo*. So the cells decide, the 95% layout is an expectation rather than a rule, and deviating from it is reported, never rejected.

3.3 parse_orders.py — an ORDER sheet becomes OrderLine rows

This file does two jobs that used to be separate: it extracts the order, **and it decides whether the sheet was an order sheet at all**. See the box at the end of this section for why those were merged.

FUNCTION	WHAT IT DOES / HOW IT WORKS
OrderLine (dataclass)	One understood row: style, color, article, sizes {label: qty, verbatim }, total, source_row (for traceable warnings), order_date, unit_price, currency, delivery_date, and size_confidence. Why price rides the line but describes the style: the sheet prints it once, on the style's own row, but a style usually has several rows (one per colourway). The line carries it because that is where the sheet puts it; the loader folds it up onto the Style, which is the level it actually describes.
_find_columns(ws, header_row, data_rows)	Purpose: for one block, which column is style, colour, article — and which are the sizes. How: call detect_size_band (that settles sizes and tail); scan the header for recognised words; work out the descriptor zone (the text columns left of the band, skipping numeric serial columns and empty spacers); apply the 95% assumption STYLE, then COLOUR, then ARTICLE; a recognised heading always overrides position ; and if the two readings disagree, raise one warning naming what was used. Why one warning and not three: John Peter deviates on two columns of every sheet they send, and three lines of noise per import is how people learn to ignore warnings.
_is_header_row(ws, r)	Does a new block start here? Column 1 is exactly S.NO/SNO or <i>contains</i> DATE, or column 2 is exactly S.NO/SNO. This is a block splitter, not a classifier. It is deliberately loose because its job is to find where a block begins <i>inside a sheet already known to be an order sheet</i> . Do not use it to decide what a document is — see the next row.
_sheet_names_itself(ws) NEW	Purpose: does this sheet announce itself as an order sheet, in so many words? How: look in the first 4 columns of the first 20 rows for a cell that is exactly S.NO, SNO, STYLE or DATE. Why exact and not a substring: GGZ-PRODUCTION heads a column <i>Cutting started DATE</i> , which a loose test matches. Treating that as an order header parses its CUTTING / FUSING / PASTING columns as sizes and reports 90,097 pieces . That one distinction is the difference between a 146-piece order and a fiction.
parse_order_sheet(ws)	Purpose: read one whole sheet; return (lines, warnings, verdict). How: find every block header; if there are none, solve for one arithmetically with find_header_row, and give up as UNKNOWN if that fails too; walk every row, re-running _find_columns at each header because each block can declare different sizes ; read descriptors, sizes, price and delivery from each data row; treat a row with no style but a colour and quantities as a continuation row that inherits the style, article and date above it; finally cross-check against any GRAND TOTAL the sheet prints.
_block_rows(header)	Inner helper. Given one header row, return exactly the rows belonging to it: from the next row down to just before the <i>next</i> header. This is what lets each block be detected on its own terms.
close_block()	Inner helper. Resets the running per-block subtotal when a new header arrives, so one block's tally never leaks into the next.

Worked example — continuation rows, John Peter Order Data

```
CLERMONT      | BURNED SUEDE | 124 WHISKY | ... | 8
CLERMONT      | BURNED SUEDE | 06 DARK BROWN | ... | 10    style repeated
               |               | 707 FOREST | ... | 39    style BLANK → inherited

the third row is still CLERMONT. without the continuation rule those 39 pieces vanish.
```

Worked example — why each block gets its own column detection

A stacked sheet restates its header for every block, and no two blocks need agree on the sizes. KJ GARMENT ORDER has **four** blocks:

```
header row 2 → rows 3..8    46 48 50 52 54 56
header row 9 → rows 10..16  25 26 27 28 | 46 48 50 52 54 56
header row 17 → rows 18..24  25 26 27 28 29 30 31 | 52 54 56 58 60 62
header row 25 → rows 26..35  48 50 52 54 56 58 60
```

handed the WHOLE sheet at once, detection sees four incompatible layouts, reconciles none of them, and falls back to a guessed band that drops columns — which is how a 347-piece order once reported 279.

The verdict — how a sheet earns the name ORDER, and why this moved here.

Classification used to be a separate pass: `import_engine._sheet_type` solved the whole sheet to answer *is this an order sheet?*, and then `parse_order_sheet` solved it all over again to extract from it. Two passes, each with its own thresholds — **and they could disagree**. They did: NIPAL-NEW PRODUCTION was declared an ORDER by the first pass and parsed to zero lines by the second, so a whole sheet vanished from the import without a single warning.

Now there is one pass and one answer:

```
named_itself = _sheet_names_itself(ws)    the header literally says S.NO / STYLE / DATE
proven       = a block hit HIGH confidence, >=2 size columns, >=2 reconciled rows

if lines AND (named_itself OR proven):    → "ORDER"
otherwise:                               → "UNKNOWN" (+ a warning saying why)
```

Requiring lines is what closes the old silent hole. A sheet that classified as ORDER and parsed to nothing used to be reported as a successful empty order. Now it is UNKNOWN, and it says so.

3.4 import_engine.py — the orchestrator

`parse_production.py` and the **PRODUCTION** branch are gone from this file. There is no `_sheet_type` any more either — its job now belongs to the parser.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>_client_key(sheet_name)</code>	Purpose: which client does this sheet belong to. Current behaviour: returns the sheet name, upper-cased and trimmed. The marker-splitting that used to strip " GARMENT ORDER" / "-PRODUCTION" (so that both KJ sheets became client KJ) is commented out — see the box below.
<code>ClientPreview</code> (dataclass)	Everything found for one client: its <code>order_lines</code> and its warnings. <i>The <code>production_cards</code> field is gone.</i>
<code>ImportPreview</code> (dataclass)	The whole dry-run result: <code>clients</code> , <code>sheet_report</code> (one row per sheet with its verdict) and <code>total_warnings</code> .
<code>ImportPreview.summary()</code>	Purpose: turn the preview into the JSON the DM actually sees. How: per client, total the pieces and collect the style names; build <code>by_style</code> so the DM sees a real size spread { 'S': 40, 'M': 53 } rather than only a collapsed total; fold each style's price and delivery up using the same precedence the loader will apply on commit (price first-wins, delivery earliest-wins); carry <code>size_confidence</code> down from the lines; count the warnings. Why mirror the loader's precedence: so that what you approve is what gets written. If the preview showed the last price and the loader wrote the first, the screen would be lying.
<code>build_preview(source)</code>	Purpose: open the workbook, read every sheet, build the full dry run. Writes nothing. How: <code>load_workbook(data_only=True)</code> — <code>data_only</code> means a formula cell yields its last-calculated value , which is why a <code>=SUM()</code> total column works; then for each sheet work out the client key and call <code>parse_order_sheet</code> once ; record the verdict in <code>sheet_report</code> ; keep the lines if ORDER, otherwise record <i>not an order sheet; skipped</i> plus whatever the parse had to say. <code>source</code> may be a path or a stream — the router hands it the upload's own spooled stream.

Worked example — all 13 sheets of GARMENT_ORDERPRODUCTION_DETAILS.xlsx

ORDER	KJ GARMENT ORDER	347 pieces	
UNKNOWN	KJ PRODUCTION	skipped	
ORDER	GGZ-GARMENT ORDER	146	
UNKNOWN	GGZ-PRODUCTION	skipped	
ORDER	NIPAL GARMENT ORDER	259	
UNKNOWN	NIPAL PRODUCTION	skipped	
ORDER	RICANO-GARMENT ORDER	150	
UNKNOWN	RICANO PRODUCTION	skipped	
ORDER	JP-GARMENT ORDER-1	1425	
ORDER	JP-GARMENT ORDER-2	838	
UNKNOWN	JP-PRODUCTION	skipped	
ORDER	NIPAL-NEW GARMENT ORDER	393	
UNKNOWN	NIPAL-NEW PRODUCTION	skipped	← used to be a 0-piece "ORDER"

Note on `_client_key` — grouping changed, totals did not.

With the marker split commented out, the preview groups by full sheet name:

before:	JP = 2263 pieces	both JP sheets grouped together
now:	JP-GARMENT ORDER-1 = 1425	
	JP-GARMENT ORDER-2 = 838	two separate rows in the preview

This affects the preview display only. On commit, `load_preview_into_order` flattens every client in the preview into the single order number the DM typed, so the pieces written are identical either way.

3.5 load_to_db.py — the idempotent writer

Runs on a **synchronous** SQLAlchemy session inside a worker thread. Re-running with the same data updates rows instead of duplicating them.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>_order_lock_key(order_no)</code>	A stable signed 64-bit lock number for one order: a blake2b hash of the order number. Why hashed in Python, not Postgres: the key is reproducible from the application side, so you can grep for it in <code>pg_locks</code> when debugging a stuck import.
<code>_acquire_order_import_lock</code>	Purpose: stop two people importing the same order at once. How: a Postgres transaction-scoped advisory lock; if someone else holds it, raise 409 with a message explaining that a large sheet takes a few minutes. A no-op off Postgres — SQLite has no advisory locks and the test suite is single-threaded. Why transaction-scoped: it releases on commit <i>and</i> on rollback, so there is nothing to leak and nothing to clean up.
<code>_get_or_create_client</code>	Find the client by name, or create it. Name is the natural key.
<code>_get_or_create_order</code>	Find this client's order by number, or create it. In the live flow the order must already exist — the commit endpoint 404s on an unknown number before any parsing. This helper serves the seed and test paths.
<code>_get_or_create_style</code>	Find the style within this order, or create it. Keyed on (order, name); on creation the code comes from <code>make_style_code(order_number, name, article)</code> . Why the article is in the code: the sticker, the traveller and the style code all have to name the same leather.
<code>_upsert_sku(...)</code>	Purpose: one row per style + colour + size. How: look for an existing SKU on that triple; if found add the quantity and report updated; otherwise build one with a deterministic code and report created; flush so the next lookup sees it and the row has an id for its order lines. Why add rather than replace: one style appears on several rows of a sheet (continuation rows, repeated colourways) and those are additional pieces of the same SKU, not a correction of it. A subtlety: the code is built from <code>style.article</code> , not the line's — so a re-imported style keeps the article it was created with and its SKUs always agree with its own code.
<code>_apply_style_commercials</code>	Purpose: move the price and delivery date the sheet printed on a row up onto the Style. How: price first-wins (a later disagreement is kept out and warned about); delivery earliest-wins , because the tightest date is the one the factory must plan to. Why neither is guessed: the same fact arrives once per colourway and the rows can disagree; a silent import would keep whichever row happened to be last, which is arbitrary.
<code>_assert_no_production_history</code>	Purpose: refuse to wipe an order the floor has already started. How: if any SKU in the order has <code>ProductionEvent</code> rows, raise instead of deleting. Why: a replace-mode re-import deletes SKUs, and deleting one that already has logged cutting or stitching destroys real production history.
<code>load_preview_into_order</code>	The entry point. (1) take the advisory lock; (2) in replace mode check for production history and refuse if any exists; (3) delete the order's existing styles, SKUs, order lines and stale rates; (4) for every line of every client: get-or-create the Style, apply its commercials, upsert one SKU per size, record a <code>SkuOrderLine</code> ; (5) stop — nothing is minted.

What this file no longer does.

A block here used to walk the parsed production cards and create `Operation` rows plus piece-rate `Rate` rows from them. It is gone with the production-sheet parser. `stats` still reports operations and rates so the API shape is unchanged, and they are now always **0**. **Wage rates are entered through the wages module**, which is the system of record for them.

`Rate` is still imported here, but only to *delete* stale rates when an order is replaced. `ProductionEvent` is still imported, but only as the history guard above.

Why there is no pre-mint here. Uploading a breakdown sheet is not a decision to produce it. The DM reviews and corrects the sheet first, and barcodes are created only at Release. The commit response carries `release_required: true` and `pieces_minted: 0` to say so.

3.6 breakdown.py — the editable table and the release gate

This is the screen the DM works on, and the gate that turns drafts into real garments. Everything here is async; only the mint itself drops to a sync session.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>list_orders()</code>	Every order, permanently — the index the breakdown screen is reached through. Each row carries its breakdown status and a <code>breakdown_url</code> . Why it matters: losing a commit response loses nothing, because every order is reachable from here.
<code>get_table(order_number)</code>	The uploaded breakdown as a table: styles, their SKUs, status and minted counts. This is the screen the DM corrects before anything is minted.
<code>update_sku(sku_id, patch)</code>	Correct one line of a DRAFT breakdown — and optionally its parent style in the same call and the same transaction. 409 once the style is RELEASED.
<code>update_style(style_id, patch)</code>	Correct a DRAFT style header on its own. 409 once released.
<code>_apply_style_patch(style, patch)</code>	Writes the allowed style fields onto the style and reports what changed. <code>needs_lining</code> is handled apart from the rest because <code>None</code> is <i>meaningful</i> there: it means “not yet answered”, which is different from “answered no”.
<code>_commit_unique_safe(style)</code>	Commit, turning a style-code unique violation into a clean 409 . Why: the pre-check in <code>_apply_style_patch</code> loses to a concurrent edit — two DMs renaming at once — so the database constraint is the real referee, and this turns its error into a sensible response instead of a 500.
<code>delete_sku(sku_id)</code>	Drop a line the sheet should not have had. DRAFT only. Why a hard delete is safe here and only here: a DRAFT SKU has no pieces, no barcodes and no history.
<code>cancel_styles(style_ids)</code>	Withdraw DRAFT styles so they stop showing on the release screen.
<code>release_styles(order_number, style_ids)</code>	The gate. Release named styles into production: mint pieces, barcodes and drawers. Checks are cheapest-first and each failure lands in rejected while the rest continue. The lining question is asked here, and this is the only place it can be asked, because after this the pieces exist. Writes one audit row per style.
<code>_order / _sku_with_style</code> <code>_assert_draft / _audit</code>	Small internal helpers: load an order by number; load a SKU with its style; raise 409 if a style is not DRAFT; write an audit-log row.
<code>_release_sync(...)</code>	The mint itself , on a synchronous session, in one transaction . Why atomic: stamping the style RELEASED and minting its pieces must happen together. A crash between them leaves either a style marked released with no garments, or garments belonging to a style that is still DRAFT.

3.7 premint.py — the mint, and the recycling drawer pool

This is the irreversible half of the module. Everything here runs only when the DM presses Release. It creates Piece rows and the barcodes that get printed and stuck onto leather.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>_sku_needs_lining(sku, db)</code>	Purpose: does this garment need a lining? How: if the DM answered at release, that answer is authoritative in both directions and short-circuits everything else; only an unanswered flag falls through to inspecting the style. Why it matters downstream: a lined jacket is not complete in its drawer until leather <i>and</i> lining are stored; a leather-only piece is complete on leather alone.
<code>_max_drawer_seq(db)</code> <code>_drawer_count(db)</code>	The highest drawer number in use, and how many drawers exist. One aggregate query each — used to number new drawers and to decide whether the pool needs topping up.
<code>_max_short_code_counter(db)</code>	The highest compact barcode number issued so far, so the next carries on from there. Why duplicated from the async repository: the importer runs on a sync session and cannot await it. Deliberately one read for the whole upload — pre-mint then counts up in Python rather than querying per piece.
<code>_build_drawer(seq, sink)</code>	Build one drawer and its barcode row, adding nothing to the session yet. Why the id is assigned here: <code>UUIDMixin.id</code> is a Python-side default that SQLAlchemy only fills at flush time, so this used to flush purely to learn the id. Assigning it up front removes that flush and matches the project rule that ids come from the application.

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>bootstrap_drawer_pool(db, size)</code>	Create the factory's initial set of empty barcoded drawers. Idempotent and additive : if the pool already holds <code>size</code> it does nothing, if fewer it tops up, and it never deletes . Safe to run from the seed script on every boot.
<code>_PoolAllocator.take()</code>	Hands out drawers: a free WAITING one if there is one, otherwise a newly minted one if the pool may still grow, otherwise None — and the piece goes on the waiting list. Why a piece can have no drawer : drawers are physical slots and there is a real, finite number of them. A piece minted when the pool is full still gets its identity and its barcode; it just cannot be stored yet.
<code>premint_order(db, order, style_ids=None)</code>	The mint . Scope to the styles the DM actually picked (omit <code>style_ids</code> and the whole order is minted); resolve lining and the caption prefix once per SKU , not per piece; create a Piece numbered 1..N within the SKU; give it two barcode rows and a drawer; bulk-insert in one transaction. Idempotent : re-running tops up rather than duplicating. No longer called from upload — it runs from <code>release_styles</code> , and that single change is what made the two-phase flow possible.
<code>allocate_waiting_pieces(db)</code>	Give drawers to pieces minted while the pool was full. Called after a piece ships (freeing its drawer), after the pool is grown, and on demand. Why : the waiting list drains itself; nobody has to remember to do it.
<code>grow_drawer_pool(db, add)</code>	Append new permanent barcoded drawers. DM/MD only. Why only on request : the pool only ever grows, and only when a human says so — that is the entire point of bounding it.
<code>drawer_pool_status(db)</code>	Pool size, how much of it is free, and how many pieces are waiting for a drawer.
<code>_caption_prefix(db, sku)</code>	The part of the sticker text that is the same for every piece of a SKU — e.g. CLERMONT · GOAT SUEDE · PINE GREEN · M; the sequence is appended per piece. Why the caption exists : the barcode carries a small unique id, but the sticker has to tell a human on the floor which garment they are holding. Resolving it once per SKU is what keeps a 1,400-piece release fast.

The two barcodes, and why there are two

PC-4F2A9C	the PRIMARY, compact code. Short enough to print small and scan reliably. Carries the order / sku / style foreign keys.
CLERMONT-PINE GREEN-M-0042	the ALIAS, long and human-readable. <code>is_alias=True</code> , carries NO foreign keys. It exists so a person can read the sticker; the scanner uses the short one.

3.8 service.py — the facade

FUNCTION	WHAT IT DOES / HOW IT WORKS
<code>preview_workbook(path)</code>	Dry-run parse of a workbook into a structured summary.
<code>commit_workbook(db, path)</code>	Parse then write a workbook into the given order, idempotently. Returns a <code>{"summary", "written"}</code> stats dict.

Both are thin wrappers kept for API stability. **The router bypasses them today** and calls `build_preview` and `load_preview_into_order` directly, so if you are tracing a live upload you will not pass through here.

4 Every endpoint, and what it costs you

All of them are Direct Manager work; MD passes as superuser. Mounted under `/api/v1`.

METHOD + PATH	HANDLER	DOES	WRITES?
POST <code>/imports/preview</code>	<code>preview_import</code>	Dry run. Validates the order number, then parses.	Nothing
POST <code>/imports/commit</code>	<code>commit_import</code>	Parses and writes styles + SKUs into that order. Idempotent, replace-mode.	DRAFT rows. pieces_minted: 0
GET <code>/imports/orders</code>	<code>list_orders</code>	The permanent order index: search, filter, breakdown_url per row.	Read-only
GET <code>/imports/breakdown/{order_number}</code>	<code>breakdown_table</code>	The breakdown as a table: styles, SKUs, status, minted counts.	Read-only
PATCH <code>/imports/breakdown/skus/{id}</code>	<code>patch_breakdown_sku</code>	Correct one DRAFT line — and its parent style in the same call and the same transaction.	SKU + Style 409 if RELEASED
PATCH <code>/imports/breakdown/styles/{id}</code>	<code>patch_breakdown_style</code>	Correct a DRAFT style header on its own.	Style 409 if RELEASED
DELETE <code>/imports/breakdown/skus/{id}</code>	<code>delete_breakdown_sku</code>	Drop a line the sheet should not have had.	Deletes DRAFT + 0 pieces only
POST <code>/imports/breakdown/{no}/cancel</code>	<code>cancel_breakdown_styles</code>	Withdraw DRAFT styles from the release screen.	Style status
POST <code>/imports/breakdown/{no}/release</code>	<code>release_breakdown_styles</code>	Mints pieces, barcodes and drawers. The irreversible one.	201 Pieces + barcodes PERMANENT

The upload guards, and where they moved

GUARD	FAILURE	WHAT IT IS FOR
filename ends .xlsx / .xlsm (and filename may be None)	400	The cheapest possible rejection, and a None filename is legal in the multipart spec.
size within settings.max_upload_mb	413	Measured against the spooled body by seeking to the end — no unbounded read into memory.
zipfile.is_zipfile(stream)	400	An .xlsx is a zip. Anything merely renamed to .xlsx is rejected before openpyxl ever parses it. This is the decompression-bomb guard.

`_save_upload` **is now** `_validated_upload`, and there is no temp file. Starlette has already spooled the body into a seekable `SpooledTemporaryFile`, and `openpyxl` reads a file-like object as happily as a path — so copying those bytes to a second file on disk bought nothing but I/O and a cleanup path to get wrong. **All three guards moved with it**; they were never a property of writing the file to disk. The stream is rewound before it is handed on.

The request bodies, and the two internal handlers

MODEL / FUNCTION	WHAT IT IS FOR
<code>StylePatch</code>	Everything a hand-made breakdown sheet routinely gets wrong on the first pass: name, article, code, gender, label, thickness, season, customer_ref, internal_ref, unit_price, currency, needs_lining. Why so wide: the sheets are built by hand from a client order and a spec sheet, and re-uploading a whole workbook to fix one thickness is not a correction workflow. <code>needs_lining</code> is here as well as on release, so a DM who realises mid-review that a style has no lining can say so without waiting for the release dialog.
<code>SkuPatch</code>	The fields one breakdown <i>line</i> can get wrong — colour, size, quantity.
<code>ReleaseStyle</code>	One style being released, plus the DM's <code>needs_lining</code> answer for it.

MODEL / FUNCTION	WHAT IT IS FOR
ReleaseRequest	The release body. <code>_one_shape</code> (a validator) accepts either a plain list of style ids or a list of <code>ReleaseStyle</code> objects; resolved normalises whichever arrived into one shape, so the service sees a single form.
StyleIdsRequest	A bare list of style ids — the body for <code>cancel</code> .
<code>_require_order(db, no)</code>	Rejects an unknown order number with 404 before a single cell is parsed.
<code>_do_preview(source)</code> <code>_do_commit_into_order(...)</code>	The two thread-pool payloads. Preview builds and summarises; commit builds, summarises, then opens a sync session and calls <code>load_preview_into_order</code> .

5 The rules that are not negotiable

Architecture

- **Router = HTTP only. Service = business logic. Repository = all DB access.** Commits belong in the service, batching the whole operation into one transaction — never scattered.
- **Parsing and bulk loading are blocking and sync.** They run in a worker thread via `run_in_threadpool` against `SessionLocal()`, so the event loop is never blocked and the proven insert ordering stays untouched. Do not port them to async to make them look tidy.
- **The upload is never copied to disk.** It is validated and read as a stream. *This replaces the old rule “the temp file never survives the request” — there is no longer a temp file to survive.*
- **Approval gates are hard, audited state transitions** — a `production_status` enum plus an `audit_log` row, never a soft boolean.

Data

- **The breakdown sheet is the production source of truth.** Production events reference the breakdown, not the raw spec.
- **Idempotence everywhere.** Re-importing the same file yields the same final numbers; re-releasing tops up rather than duplicating; parsing the same file always produces identical output.
- **Codes are deterministic,** built from the order, style, colour, size and article — never from a counter that could drift.
- **Sizes are stored verbatim.** Case and spacing are folded, nothing else. XXL/54 stays XXL/54, because whatever the client wrote is what the floor reads back.
- **The importer reads order sheets only.** Anything else is reported and skipped; it is never guessed at.

Parsing

- **No size vocabulary, ever.** The band is solved against the printed total. Adding a word list back is how a new country's order silently ships short.
- **A band must carry a positive quantity.** Zero reconciles with zero and proves nothing.
- **Footers stay out of the column profile.** They are not counter-examples, they are footers — but a caption inside the band will disqualify a real size column if you let it into the sample.
- **MEDIUM is never silently promoted.** A sheet that prints a total which does not add up names the rows that disagree.

What the parser has actually been proven against

WORKBOOK	RESULT
GARMENT_ORDERPRODUCTION_DETAILS.xlsx	13 sheets: 7 ORDER, 6 skipped. KJ 347 · GGZ 146 · NIPAL 259 · RICANO 150 · JP-1 1425 · JP-2 838 · NIPAL-NEW 393.
johnpeter.xlsx	1425 pieces across 17 styles, from a 16-column band mixing S M L XL with 38...60.
John_Peter_Order_Data_Consolidated.xlsx	Italian headers (Modello Materiale Colore), price before the total — read correctly, with both layout warnings raised.
KairoX_Breakdown_Sheet_TEMPLATE.xlsx	TOTAL 10, PRICE 11, DELIVERY 12; € 78,50 cif → 78.50 EUR. No warnings.
SAMPLE COSTING SHEET FOR SIR.xlsx	21 sheets, all rejected, 0 pieces. Previously came back as 16,649,667 pieces.

If you remember one thing

Everything before Release is reversible. Everything after it is printed on leather.